ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΜΑΚΕΔΟΝΙΑΣ

ΣΧΕΔΙΑΣΜΟΣ ΚΑΙ ΥΛΟΠΟΙΗΣΗ 8 BIT CPU ΓΙΑ ΤΗΝ ΠΛΑΚΕΤΑ ALTERA-DE2-115

ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΟΝΟΜΑ: ΓΕΩΡΓΙΟΣ ΠΑΠΑΜΙΧΑΗΛ

2026

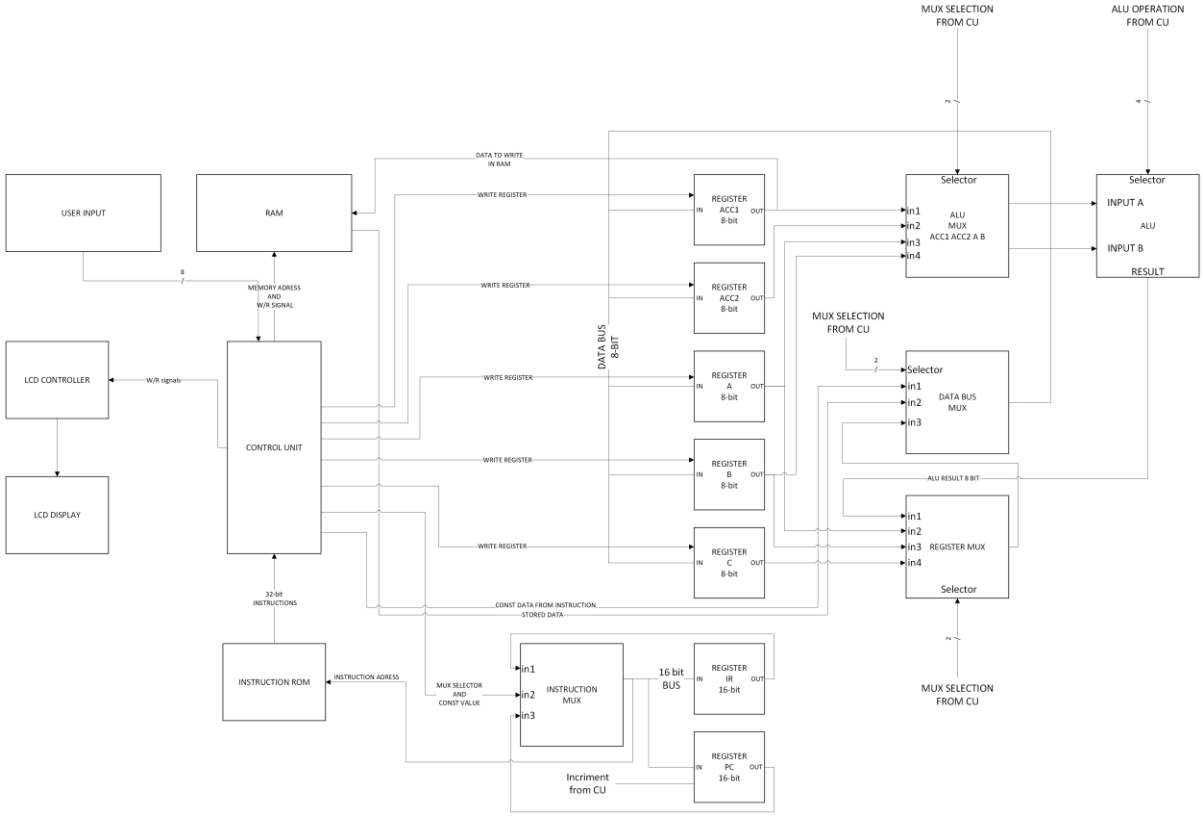
1. Εισαγωγή 3
2. Αρχιτεκτονική 4
3. Εντολές 6
4. Καταχωρητές 11
5. ALU 12
6. Διαδρομή Δεδομένων (Datapath) 13
7. Control Unit 15
8. Assembler 17
9. Παραδείγματα κώδικα 20
10. Περιορισμοί 26

1. Εισαγωγή

Το παρόν έγγραφο παρουσιάζει την αρχιτεκτονική ενός προσαρμοσμένου 8-bit επεξεργαστή και παρέχει βασικές πληροφορίες σχετικά με τη δομή και τη λειτουργία του. Ο επεξεργαστής χρησιμοποιεί 256 διευθύνσεις RAM, καθεμία από τις οποίες αποθηκεύει ένα byte, από τη διεύθυνση 0x00 έως τη διεύθυνση 0xFF. Οι εντολές αποθηκεύονται σε μνήμη προγράμματος μέγιστης χωρητικότητας 65.536 (2^{16}) διευθύνσεων (ROM) και αποτελούνται από OP_CODE + OPERANDS 32-bit, το οποίο μεταφέρεται απευθείας στη μονάδα ελέγχου (Control Unit) για αποκωδικοποίηση και εκτέλεση. Η αρχιτεκτονική περιλαμβάνει ένα σύνολο καταχωρητών 8-bit για την αποθήκευση δεδομένων και δύο καταχωρητές 16-bit για τη διαχείριση της εκτέλεσης των εντολών, καθώς και ένα σύστημα flags για την ένδειξη καταστάσεων όπως το carry και το zero. Οι αριθμητικές και λογικές πράξεις υλοποιούνται από την ALU, η οποία υποστηρίζει βασικές λειτουργίες επεξεργασίας δεδομένων. Η επικοινωνία μεταξύ των υπομονάδων του επεξεργαστή πραγματοποιείται μέσω διαύλων δεδομένων και διευθύνσεων, ενώ η Control Unit διαχειρίζεται τον κύκλο εκτέλεσης των εντολών μέσω των σταδίων fetch, decode και execute.

Σκοπός του εγγράφου είναι να αναδείξει τις θεμελιώδεις μονάδες του επεξεργαστή, ξεκινώντας από τα υποσυστήματά του και καταλήγοντας στη συνολική αρχιτεκτονική.

2. Αρχιτεκτονική



Η αρχιτεκτονική του επεξεργαστή αποτελείται από βασικές μονάδες που συνεργάζονται για την εκτέλεση των εντολών. Το διάγραμμα παρουσιάζει τη συνολική ροή δεδομένων και ελέγχου μέσα στην CPU.

Καταχωρητές (Registers)

Συνολικά υπάρχουν 8 καταχωρητές, εκ των οποίων ο προγραμματιστής μπορεί να χρησιμοποιήσει μόνο τους 5, καθώς οι υπόλοιποι τρεις είναι ο IR, ο PC και ο καταχωρητής των Flags.

- **Γενικής χρήσης:** 2 καταχωρητές (A και B), 8-bit, για προσωρινή αποθήκευση δεδομένων.
- **Ειδικής χρήσης:** 3 καταχωρητές (ACC1, ACC2, C), 8-bit, που χρησιμοποιούνται από την ALU για υπολογισμούς και από τον LCD controller για εκτύπωση.
- **Εσωτερικής χρήσης:** 2 καταχωρητές, ο IR (Instruction Register) και ο PC (Program Counter), 16-bit έκαστος για την αποθήκευση της τρέχουσας εντολής και της διεύθυνσής της.
- **Flags:** Καταχωρητής 8-bit, όπου αξιοποιούνται τα 2 bit ως Carry και Zero flag.

ALU (Arithmetic Logic Unit)

Υποστηρίζει 8 βασικές πράξεις: Πρόσθεση, Αφαίρεση, OR, AND, XOR, NOT, Left Shift, Right Shift.

Μνήμη

- **ROM:** Αποθηκεύει τις εντολές του προγράμματος. Η Control Unit ανακτά απευθείας τις εντολές από τη ROM χρησιμοποιώντας τη διεύθυνση που δίνει ο Program Counter (PC) Στον IR. Κάθε εντολή έχει μέγεθος 32-bit.
- **RAM:** 256 διευθύνσεις των 8-bit (1 byte ανά διεύθυνση), για προσωρινή αποθήκευση δεδομένων.

Μονάδες Ελέγχου και Εισόδου/Εξόδου

- **Μονάδα Ελέγχου Εντολών:** Παράγει τα σήματα ελέγχου για την εκτέλεση των εντολών.
- **Μονάδα Εγγραφής σε Οθόνη LCD:** Επιτρέπει την εμφάνιση αποτελεσμάτων. Ο χαρακτήρας που θα τυπωθεί ανακτάται από τον καταχωρητή C.
- **8-bit Είσοδος Χρήστη:** Δέχεται δεδομένα από τον χρήστη μέσω 8 διακοπτών που αντιστοιχούν σε τιμές 0 ή 1 για κάθε bit. Ένα κουμπί χρησιμοποιείται για να στείλει την τρέχουσα τιμή στη Control Unit, η οποία την επεξεργάζεται σύμφωνα με την εντολή του προγράμματος.

Γενική Λειτουργία

Η εκτέλεση κάθε εντολής γίνεται σε τρία βασικά βήματα:

1. Το περιεχόμενο της διεύθυνσης του PC φορτώνεται στον IR και στη συνέχεια ο PC αυξάνεται κατά 1.
2. Η Μονάδα Ελέγχου αποκωδικοποιεί την εντολή και ενεργοποιεί τα κατάλληλα σήματα για την ALU και τους καταχωρητές.
3. Η ALU εκτελεί την πράξη και ενημερώνει τους καταχωρητές και τα flags.

3. Εντολές

Ο επεξεργαστής υποστηρίζει διάφορες εντολές, οι οποίες χωρίζονται σε κατηγορίες ανάλογα με τη λειτουργία τους: **μεταφοράς** δεδομένων, **αριθμητικές/λογικές και ολίσθησης (shift)**, **σύγκρισης**, **άλματος**, **εισόδου/εξόδου** και **ελέγχου** προγράμματος.

Η κωδικοποίηση των εντολών χωρίζεται σε 4ης τρόπους:

1. Χωρίς παραμέτρους

8-bit OP	Τα υπόλοιπα είναι κενά

31 0

2. Μίας παραμέτρου

8-bit OP	8-bit για τον καταχωρητή, τιμής από την εντολή ή διεύθυνσης RAM	Τα υπόλοιπα είναι κενά

31 0

3. Μίας παραμέτρου για άλματα

8-bit OP	16-bit για την διεύθυνση άλματος	Τα υπόλοιπα είναι κενά

31 0

4. Δύο παραμέτρων

8-bit OP	8-bit για τον καταχωρητή, τιμής από την εντολή ή διεύθυνσης RAM	8-bit για τον καταχωρητή, τιμής από την εντολή ή διεύθυνσης RAM	Τα υπόλοιπα είναι κενά

31 0

1. Μεταφορά Δεδομένων

Οι αριθμοί μπορούν να δοθούν σε δεκαδική μορφή (π.χ. `100`), δεκαεξαδική (π.χ. `xA0`) ή δυαδική (π.χ. `b00101010`) και να κυμαίνονται από 0 έως 255. Για να οριστεί μια μεταβλητή στη RAM, στην αρχή του προγράμματος καθορίζεται με τη μορφή `*onoma: διεύθυνση`, π.χ. `*d1 : xAA`, όπου η τιμή μπορεί να δοθεί επίσης σε δεκαδικό, δυαδικό ή δεκαεξαδικό σύστημα (π.χ. `200` ή `b11111111`).

MOV Register, VALUE_IN_CODE. ➔ Μεταφορά τιμής από τον κώδικα στον **καταχωρητή**. Register: ACC1, ACC2, A, B, C.

LD Register, *<**διεύθυνση** RAM>. ➔ Μεταφορά της τιμής της **διεύθυνσης** RAM στον καταχωρητή. Register: ACC1, ACC2, A, B, C.

ST *<**διεύθυνση** RAM>, **Register**. ➔ Μεταφορά **περιεχομένου** καταχωρητή στη θέση της **διεύθυνσης** RAM. Register: ACC1, A, B, C.

MOVR Register1, Register2. ➔ Ανριγραφή του Register2 στον Register1. Register: A, B, C.

Στον assembler όλες οι **εντολές μετακίνησης** ονομάζονται MOV και **αναγνωρίζει** από μόνος του το source και destination των **δεδομένων**. Η κωδικοποίηση τους ακολουθεί την μορφή **4. Δύο παραμέτρων**

2. Αριθμητικές / Λογικές Πράξεις

Οι αριθμητικές και λογικές πράξεις διακρίνονται σε εντολές μιας παραμέτρου και δύο παραμέτρων. Στις εντολές μιας παραμέτρου, η πράξη εκτελείται χρησιμοποιώντας τους καταχωρητές **ACC1** και **ACC2** και το αποτέλεσμα αποθηκεύεται στον αναφερόμενο καταχωρητή, ενώ για αυτές τις εντολές μπορούν να χρησιμοποιηθούν όλοι οι καταχωρητές. Στις εντολές δύο παραμέτρων, η πράξη εκτελείται μεταξύ των δύο αναφερόμενων καταχωρητών και το αποτέλεσμα αποθηκεύεται στον πρώτο καταχωρητή, με την προϋπόθεση ότι οι καταχωρητές είναι μόνο **ACC1, ACC2, A ή B**.

Όλες οι αριθμητικές και λογικές πράξεις επηρεάζουν τα **flags** του καταχωρητή κατάστασης. Συγκεκριμένα, ενημερώνεται το **Carry flag** σε περίπτωση υπερχείλισης ή borrow και το **Zero flag** όταν το αποτέλεσμα της πράξης είναι μηδέν.

Οι παρακάτω εντολές ακολουθούν την κωδικοποίηση **2. Μίας παραμέτρου**

ADD Register ➔ Αποθήκευση του **αποτελέσματος της πρόσθεσης** των ACC1 + ACC2 στον Register καταχωρητή.

SUB Register ➔ Αποθήκευση του **αποτελέσματος της αφαίρεσης** των ACC1 - ACC2 στον Register καταχωρητή.

OR Register ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** ACC1 | ACC2 στον Register καταχωρητή.

AND Register ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** ACC1 & ACC2 στον Register καταχωρητή.

NOT Register ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** ~ACC1 στον Register καταχωρητή.

XOR Register ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** ACC1 xor ACC2 στον Register καταχωρητή.

INC Register ➔ Αύξηση κατά 1 του ACC1 καταχωρητή.

DEC Register ➔ Μείωση κατά 1 του ACC1 καταχωρητή.

Οι παρακάτω εντολές ακολουθούν την κωδικοποίηση **4. Δύο παραμέτρων**

ADD Register1, Register2 ➔ Αποθήκευση του **αποτελέσματος της πρόσθεσης** των Register1 + Register2 στον Register1 καταχωρητή.

SUB Register1, Register2 ➔ Αποθήκευση του **αποτελέσματος της αφαίρεσης** των Register1 - Register2 στον Register1 καταχωρητή.

OR Register1, Register2 ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** Register1 | Register2 στον Register1 καταχωρητή.

AND Register1, Register2 ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** Register1 & Register2 στον Register1 καταχωρητή.

NOT Register1, Register1 ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** ~Register1 στον Register1 καταχωρητή.

XOR Register1, Register2 ➔ Αποθήκευση του **αποτελέσματος της λογικής πράξης** Register1 xor Register2 στον Register1 καταχωρητή.

INC Register1, Register1 ➔ Αύξηση κατά 1 του Register1 καταχωρητή.

DEC Register1, Register1 ➔ Μείωση κατά 1 του Register1 καταχωρητή.

SHL Register, N. ➔ **Ολίσθηση (Shift)** αριστερά N **φορές** του Register καταχωρητή.

SHR Register, N. ➔ **Ολίσθηση (Shift)** δεξιά N **φορές** του Register καταχωρητή.

4. Σύγκριση

CMP R1, R2. ➔ Συγκρίνει δύο τιμές και ενημερώνει τα flags χωρίς να αποθηκεύει αποτέλεσμα. Οι παράμετροι **R1** και **R2** μπορούν να είναι καταχωρητές, άμεσες τιμές από την εντολή ή τιμές από τη RAM. Οι άμεσες τιμές μπορούν να δοθούν σε δεκαδική (π.χ. `100`), δεκαεξαδική (π.χ. `xA0`) ή δυαδική μορφή (π.χ. `b00101010`), ενώ οι τιμές από τη RAM δηλώνονται με το σύμβολο * πριν από το όνομα της μεταβλητής. Η κωδικοποίηση τους ακολουθεί την μορφή **4. Δύο παραμέτρων**

Το **R1 μετακινείται** στον ACC1 και το **R2** στον ACC2.

Στη συνέχεια, εκτελείται η πράξη ACC1 - ACC2, η οποία **ενημερώνει** τα flags.

5. Άλματος

- <TAG>: Το TAG ορίζεται από τον χρήστη στον κώδικα και αναγνωρίζεται από τον assembler μέσω του συμβόλου - πριν από το όνομά του (π.χ. -loop). Ο assembler συσχετίζει το TAG με τη διεύθυνση της επόμενης εντολής, ώστε να μπορεί να χρησιμοποιηθεί ως προορισμός σε εντολές διακλάδωσης (jump). Η κωδικοποίηση τους ακολουθεί την μορφή **3. Μίας παραμέτρου για άλματα**

JMP <TAG> ➔ Άμεσο άλμα στη διεύθυνση <TAG>.

JA <TAG> ➔ Άλμα αν **carry/borrow flag** είναι 0 (δηλαδή **R1 > R2)**.

JB <TAG> ➔ Άλμα αν **carry/borrow flag** είναι 1 (δηλαδή **R1 < R2)**.

JE <TAG> ➔ Άλμα αν **zero flag** είναι 1 (δηλαδή **R1 = R2)**.

JAE <TAG> ➔ Άλμα αν **carry/borrow flag** είναι 0 ή **zero flag** είναι 1 (δηλαδή **R1 ≥ R2)**.

JBE <TAG> ➔ Άλμα αν **carry/borrow flag** είναι 1 ή **zero flag** είναι 1 (δηλαδή **R1 ≤ R2)**.

JC <TAG> ➔ Άλμα αν το **carry flag** είναι 1.

JZ <TAG> ➔ Άλμα αν το **zero flag** είναι 1.

6. Είσοδος / Έξοδος

IN Register ➔ Λήψη 8-bit δεδομένων από τον χρήστη στον **Register** καταχωρητή (Στην τρέχουσα υλοποίηση η είσοδος έρχεται από τους 8 διακόπτες πάνω στο Board). Η κωδικοποίηση ακολουθεί την μορφή της 2. Μ**ία παραμέτρου.**

PRT.➔ Εμφάνιση στην LCD του δεδομένου που βρίσκεται στον καταχωριτή C. Η κωδικοποίηση ακολουθεί την μορφή **1.** Χ**ωρίς παραμέτρου.**

CLR ➔ Εκκαθάριση LCD οθόνης. Η κωδικοποίηση ακολουθεί την μορφή **1.** Χ**ωρίς παραμέτρου.**

HLT ➔ Διακοπή εκτέλεσης. Η κωδικοποίηση ακολουθεί την μορφή **1.** Χ**ωρίς παραμέτρου.**

4. Καταχωρητές

Ο επεξεργαστής διαθέτει διάφορους **καταχωρητές**, χωρισμένους σε γενικής, ειδικής και εσωτερικής χρήσης, καθώς και **καταχωρητές flags**.

Τύπος	Όνομα	Μέγεθος	Λειτουργία
Γενικής χρήσης	A, B	8-bit	Αποθήκευση προσωρινών δεδομένων για υπολογισμούς και μεταφορά τους.
Ειδικής χρήσης	ACC1	8-bit	Αποθήκευση προσωρινών δεδομένων. Αποθήκευση της τιμής που θα μεταφερθεί στη RAM.
Ειδικής χρήσης	ACC2	8-bit	Αποθήκευση προσωρινών δεδομένων για υπολογισμούς.
Ειδικής χρήσης	C	8-bit	Αποθήκευση προσωρινών δεδομένων για μεταφορά. Εκτύπωση δεδομένων στην οθόνη LCD.
Εσωτερικής χρήσης	IR (Instruction Register)	16-bit	Αποθήκευση της τρέχουσας εντολής προς εκτέλεση.
Εσωτερικής χρήσης	PC (Program Counter)	16-bit	Περιέχει τη διεύθυνση της επόμενης εντολής.
Flags	SR (Status Register)	2-bit	Παρακολούθηση αποτελεσμάτων της ALU για Carry και Zero (μηδενικό αποτέλεσμα). Στην αφαίρεση, το Carry Flag χρησιμοποιείται ως Borrow (δανεικό).

5. ALU

Η ALU αποτελεί τη βασική αριθμητική και λογική μονάδα του επεξεργαστή και είναι υπεύθυνη για την εκτέλεση όλων των πράξεων που απαιτούν υπολογισμό. Λειτουργεί αποκλειστικά με 8-bit δεδομένα και χρησιμοποιεί τους καταχωρητές **A, B, ACC1, ACC2 και C** ως πηγές και προορισμούς τιμών.

Υποστηριζόμενες Πράξεις

Η ALU υλοποιεί τις ακόλουθες 8 βασικές λειτουργίες:

ADD, SUB, OR, AND, XOR, NOT, SHL, SHR

Μετά από κάθε πράξη, η ALU ενημερώνει τους δύο καταχωρητές flags:

Carry (C): Ενεργοποιείται όταν υπάρχει υπερχείλιση σε πρόσθεση ή δανεισμός σε αφαίρεση.

Zero (Z): Ενεργοποιείται όταν το αποτέλεσμα της πράξης είναι 0.

Ροή Λειτουργίας

Η διαδικασία εκτέλεσης μιας πράξης ακολουθεί τα παρακάτω βήματα:

1. **Λήψη Εντολής:** Η Μονάδα Ελέγχου (Control Unit) αποκωδικοποιεί την εντολή και στέλνει το κατάλληλο **Opcode** (κωδικό επιχείρησης) στην ALU.

2. **Επιλογή Δεδομένων:** Η ALU λαμβάνει τις τιμές εισόδου από τους αντίστοιχους καταχωρητές (**A, B** ή **ACC1/ACC2**). Η επιλογή των σωστών δεδομένων γίνεται μέσω ενός **πολυπλέκτη** (Multiplexer - MUX).

3. **Εκτέλεση:** Η ALU εκτελεί την αριθμητική ή λογική πράξη που ορίζεται από το Opcode.
4. **Αποθήκευση:** Το αποτέλεσμα της πράξης μεταφέρεται και αποθηκεύεται στον προκαθορισμένο καταχωρητή προορισμού.
5. **Ενημέρωση Κατάστασης:** Γίνεται αυτόματη ενημέρωση των **Flags** (Carry και Zero), ώστε ο επεξεργαστής να γνωρίζει την κατάσταση του αποτελέσματος για τις επόμενες εντολές.

Χαρακτηριστικά

- **Χρονισμός:** Όλες οι πράξεις ολοκληρώνονται σε **2 κύκλους ρολογιού (clock cycles)**, εξασφαλίζοντας σταθερή ταχύτητα εκτέλεσης.
- **Συνεργασία Μονάδων:** Η ALU συνεργάζεται στενά με τη **Μονάδα Ελέγχου (Control Unit)**, η οποία καθορίζει την επιθυμητή πράξη μέσω των **σημάτων ελέγχου (control signals)**.

- **Άμεση Διαθεσιμότητα:** Τα αποτελέσματα των πράξεων είναι διαθέσιμα στους καταχωρητές ώστε να χρησιμοποιηθούν **άμεσα** από τις επόμενες εντολές του προγράμματος.

6. Διαδρομή Δεδομένων (Datapath)

1. Πολυπλέκτες (Multiplexers):

Ονομασία	Περιγραφή / Λειτουργία	Είσοδοι (Sources)
MUX_DATA (MUX1)	Επιλογέας Εγγραφής Καταχωρητών. Καθορίζει τι θα γραφτεί στους A, B, C, ACC1, ACC2.	1. Immediate Value (από instruction), 2. RAM Data Out, 3. MUX_INTERNAL Output
MUX_INTERNAL (MUX2)	Επιλογέας Εσωτερικής Μεταφοράς. Επιτρέπει τη μεταφορά μεταξύ καταχωρητών ή το αποτέλεσμα της ALU.	1. A, 2. B, 3. C, 4. ALU Result
MUX_ALU_IN (MUX3)	Επιλογέας Τελεστέων ALU. Διπλός πολυπλέκτης που τροφοδοτεί τις εισόδους A και B της ALU.	1. ACC1, 2. ACC2, 3. A, 4. B
MUX_ADDR (MUX4)	Επιλογέας Διεύθυνσης/Ροής. Ελέγχει την ανανέωση των PC και IR.	1. IR, 2. PC, 3. Jump Address (από instruction)

2. Μνήμες & Διαχείριση Δεδομένων

Instruction Fetch (ROM): Η διαδικασία ανάκτησης εντολών βασίζεται στη συνεργασία του PC και του IR, με την εξής σειρά λειτουργίας:

- **Λειτουργία PC & IR:** Σε κάθε κύκλο, ο **PC** (Program Counter) στέλνει την τρέχουσα διεύθυνση στον καταχωρητή **IR**. Αμέσως μετά τη μεταφορά της τιμής, ο PC αυξάνει την τιμή του κατά 1 (μέσω του εσωτερικού **incrementor**), ώστε να είναι προετοιμασμένος για την επόμενη εντολή.
- **Διευθυνσιοδότηση ROM:** Ο **IR** είναι αυτός που κρατά τη διεύθυνση σταθερή και την παρέχει στην είσοδο της **ROM** μέσω του **MUX4**.
- **Απευθείας Σύνδεση με CU:** Η ROM, διαβάζοντας τη διεύθυνση από τον IR, παρέχει το **32-bit OpCode** απευθείας στη Μονάδα Ελέγχου (**Control Unit**). Έτσι, η αποκωδικοποίηση ξεκινά αμέσως, χωρίς να χρειάζεται να αποθηκευτεί το OpCode σε κάποιον ενδιάμεσο καταχωρητή δεδομένων.
- **Διαχείριση Αλμάτων (Jumps):** Ο **MUX4** επιτρέπει στο Control Unit να παρακάμψει την κανονική ροή, δίνοντας στον PC μια άμεση τιμή/διεύθυνση για την εκτέλεση αλμάτων στο πρόγραμμα.

Data Storage (RAM): * **Address In:** Η διεύθυνση παρέχεται από το **CU Address Field** (το τμήμα της εντολής που ορίζει τη θέση μνήμης).

- **Data In:** Δέχεται δεδομένα αποκλειστικά από τον **ACC1** (Accumulator 1).

LCD Output: Ο καταχωρητής **C** λειτουργεί ως **Output Buffer**, συνδεδεμένος απευθείας με τον ελεγκτή της οθόνης.

3. Σήματα Ελέγχου (Control Signals)

Για να λειτουργήσουν τα παραπάνω, η Μονάδα Ελέγχου (CU) πρέπει να παράγει τα εξής σήματα:

- `REG_WRITE`: Επιλέγει ποιος καταχωρητής θα ενεργοποιηθεί για εγγραφή με ένα σήμα των 8-bit.
- `ALU_SEL`: Επιλέγει την πράξη της ALU με ένα σήμα των 4-bit.
- `MUX_SEL_1/2/3/4`: Επιλέγουν την κατάλληλη είσοδο για κάθε πολυπλέκτη με ένα σήμα των 4-bit.

7. Control Unit

Η Μονάδα Ελέγχου (CU) αποτελεί το τμήμα του επεξεργαστή που συντονίζει τη λειτουργία όλων των υποσυστημάτων. Υλοποιεί μια **Μηχανή Πεπερασμένων Καταστάσεων (FSM)**, η οποία μεταφράζει το Opcode της εντολής σε σήματα ελέγχου.

Ονοματολογία Σημάτων Ελέγχου

Πριν την ανάλυση των καταστάσεων, ορίζεται η τυποποίηση των σημάτων που χρησιμοποιούνται στο Datapath:

- **Κατάληξη `_ld` (Load Enable):** Ενεργοποιεί την εγγραφή σε καταχωρητή (π.χ. `A_ld`, `ir_ld`). Όταν το σήμα είναι **'1'**, ο καταχωρητής αποθηκεύει τα δεδομένα της εισόδου του στην επόμενη ακμή του ρολογιού. Ο καταχωριτής επιλέγετε από το πεδίο `REG_WRITE.`
- **Κατάληξη `_sel` (Selector):** Ελέγχει τους πολυπλέκτες (π.χ. `data_bus_sel`). Καθορίζει ποια από τις εισόδους του πολυπλέκτη θα οδηγηθεί στην έξοδο.
- **Κατάληξη `_wr` (Write Enable):** Ενεργοποιεί την εγγραφή στη μνήμη **RAM** (`ram_wr`).
- **Κατάληξη `_inc` (Increment):** Ενεργοποιεί την αυτόματη αύξηση της τιμής του **PC** κατά 1 (`pc_inc`).

Λειτουργία Πολυπλεκτών (Multiplexer Control)

Η CU ελέγχει τη ροή των δεδομένων μέσω των εξής τιμών στους επιλογείς:

- **data_bus_sel (2 bits):**
  - `01`: Είσοδος δεδομένων από τη **RAM**.
  - `10`: Είσοδος άμεσης τιμής (**Immediate**) από τη ROM.
  - `00`: Είσοδος από τον εσωτερικό πολυπλέκτη καταχωρητών.
- **alu_mux_selA / B (2 bits):** Επιλογή πηγής για τις εισόδους της ALU.
  - `00`: ACC1 | `01`: ACC2 | `10`: A | `11`: B.
- **mux_sel (2 bits):** Επιλογή τιμής προς εγγραφή στο κύριο bus.
  - `11`: Αποτέλεσμα **ALU** (ALU_OUT).
  - `00-10`: Τιμή από υπάρχοντα καταχωρητή (A, B, C).

Κύκλος Εντολής (FSM States)

1. **FETCH (1 & 2):**
   a. `ir_ld <= '1'`: Μεταφορά της διεύθυνσης από τον PC στον IR.
   b. `pc_inc <= '1'`: Αύξηση της τιμής του PC.
   c. `pc_mux_sel <= "01"`: Επιλογή του PC για τη διευθυνσιοδότηση της ROM.

2. **DECODE:**
    a. Ανάγνωση των bits 31-24 (Opcode) και ρύθμιση των Selectors για την προετοιμασία των διαδρομών.
3. **EXECUTE:**
    a. Ενεργοποίηση των σημάτων **Load Enable** (`_ld`) για την οριστικοποίηση της αποθήκευσης των δεδομένων στους καταχωρητές προορισμού.

Λογική Διασύνδεσης (I/O & Handshaking)

- **Εντολή Εισόδου (IN):** Η CU παραμένει σε κατάσταση αναμονής μέχρι η είσοδος `confirm` να γίνει '1', διασφαλίζοντας τον συγχρονισμό με τον χρήστη πριν την ενεργοποίηση του `_ld`.
- **Εντολές LCD (PRT/CLR):** Η CU ελέγχει το σήμα `lcd_in` (Ready/Busy). Αν είναι '1', εκτελείται η εντολή ενεργοποιώντας τα σήματα `write_lcd` ή `clr_lcd`.
- **Λογική Αλμάτων (Jumps):** Η CU αξιολογεί τα σήματα `carry` και `zero`. Αν η συνθήκη επαληθεύεται, ενεργοποιεί το `pc_ld` και θέτει `pc_mux_sel <= "10"` για τη φόρτωση της νέας διεύθυνσης.

Σύνολο Εντολών (Instruction Set Architecture - ISA)

Ο επεξεργαστής εκτελεί εντολές **32-bit**. Το πεδίο **Opcode** καταλαμβάνει τα bits 31-24, ενώ τα υπόλοιπα bits ορίζουν παραμέτρους και διευθύνσεις.

Κατηγορία	Εντολή	Opcode (Hex)
Μεταφορά	MOV	0A
	LD	0B
	ST	0C
	MVR	0D
ALU	ADD / SUB	11 / 12
	ADD2 / SUB2	21 / 22
	OR / AND	13 / 14
	SHR / SHL	19 / 1A
Έλεγχος	JMP	F0
	JE / JZ	F1 / F7
	JA / JB	F2 / F3
Σύστημα	IN	FC
	PRT	FD

	CLR	FE
	HLT	FF

8. Assembler

Ο Assembler λειτουργεί σε δύο στάδια (two-pass assembler):

Pass 1 – Ανάλυση & Συλλογή Συμβόλων

- Εντοπίζει labels.
- Αποθηκεύει τις διευθύνσεις τους σε έναν πίνακα συμβόλων (symbol table).
- Ελέγχει τη σύνταξη κάθε εντολής.

Pass 2 – Μετάφραση σε Machine Code

- Μεταφράζει κάθε εντολή στο αντίστοιχο opcode.
- Μετατρέπει καταχωρητές, τιμές και διευθύνσεις σε 8-bit fields.
- Αντικαθιστά labels με τις τελικές διευθύνσεις.
- Δημιουργεί το τελικό 32-bit instruction word:

Parsing & Tokenization

Ο Assembler αναλύει κάθε γραμμή Assembly και την μετατρέπει σε tokens:

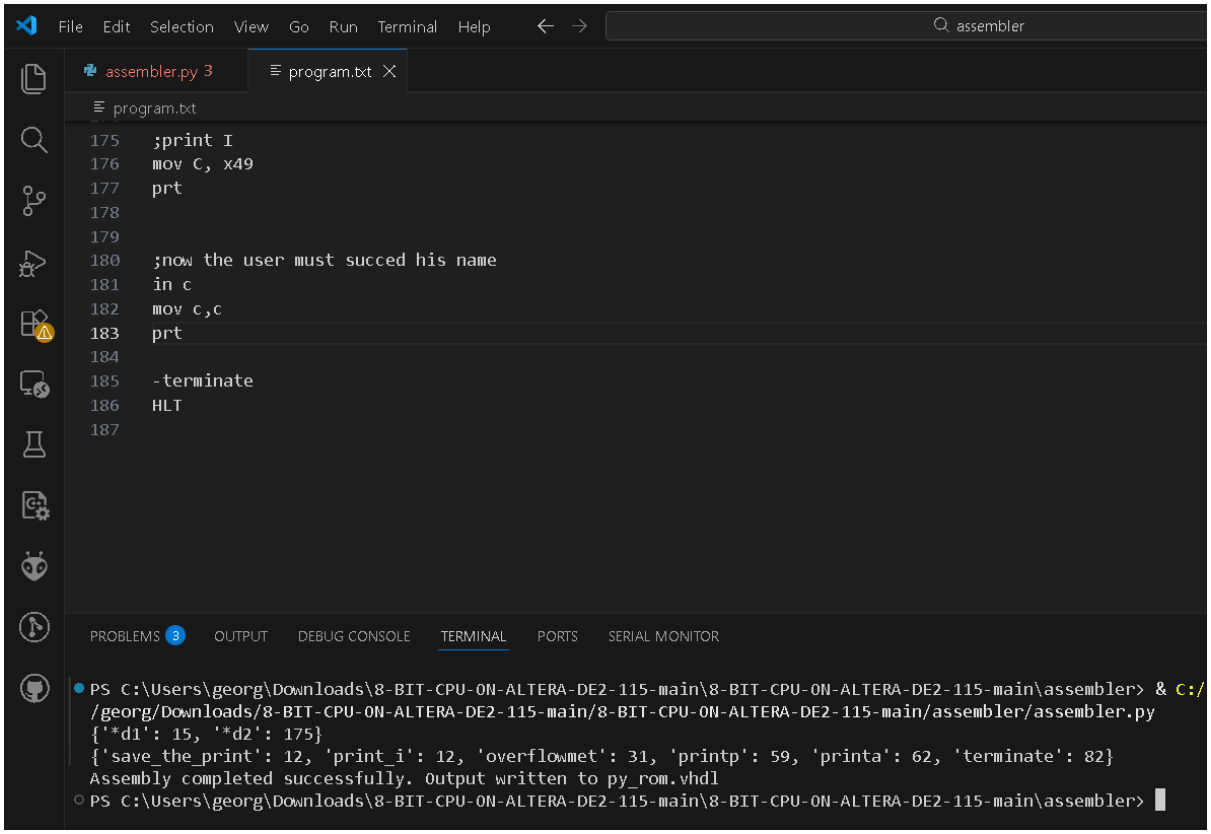
Παράδειγμα:

`ADD  A, B`
Tokenization -> `["ADD", "A", "B"]`

Υποστηρίζει επίσης:

- Σχόλια (`; text`)
- Labels (π.χ. - `loop:`)
- Σταθερές σε δεκαδικό bin ή hex (20, b00100100, x14)

Οι κενές γραμμές αγνοούνται.

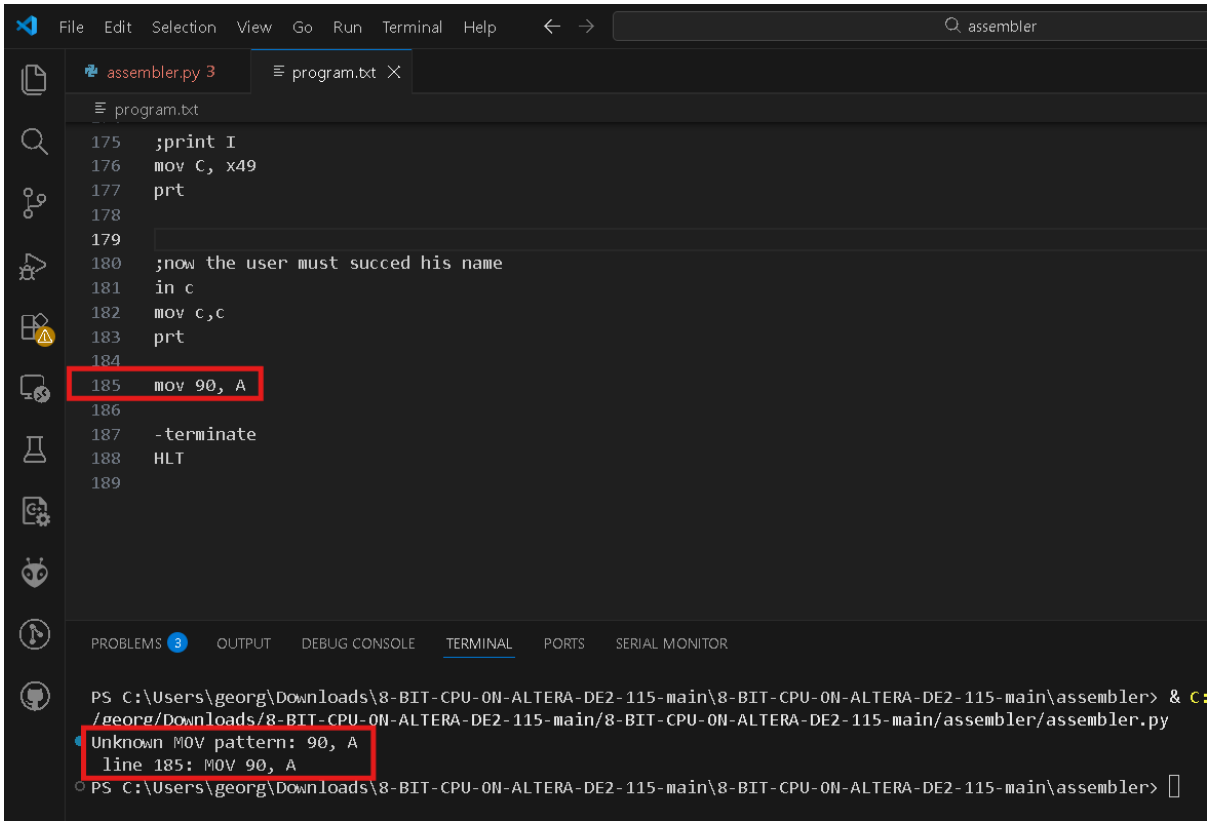


Error Reporting

Ο Assembler εντοπίζει:

- άγνωστη εντολή
- λάθος αριθμό operands
- μη έγκυρο όνομα register
- overflow immediate (> 255)
- label που δεν έχει οριστεί

Αναφέρει το σφάλμα με αριθμό γραμμής, π.χ.:



9. Παραδείγματα κώδικα

Στην ενότητα αυτή παρουσιάζεται ένα πλήρες παράδειγμα προγράμματος σε Assembly που υλοποιήθηκε για τον επεξεργαστή. Το πρόγραμμα εκτελεί σύνθετες λειτουργίες όπως διαχείριση μνήμης, αριθμητικές πράξεις, εντολές άλματος (jumps) και χρήση της οθόνης για την εκτύπωση του ονόματος "GIORGO PAPAMICHAIL".

Συγκεκριμένα, ο κώδικας:

- Αποθηκεύει και ανακτά δεδομένα από τη μνήμη RAM (χρήση μεταβλητών `*d1`, `*d2`).
- Χρησιμοποιεί εντολές ολίσθησης (Shift) και λογικές πράξεις (AND, XOR) για τη διαμόρφωση χαρακτήρων.
- Υλοποιεί βρόχους (loops) για την επαναληπτική εκτύπωση χαρακτήρων (π.χ. "PAPA").
- Διαχειρίζεται τα Flags (Carry, Overflow) για τη λήψη αποφάσεων ροής.

Κώδικας Προγράμματος:

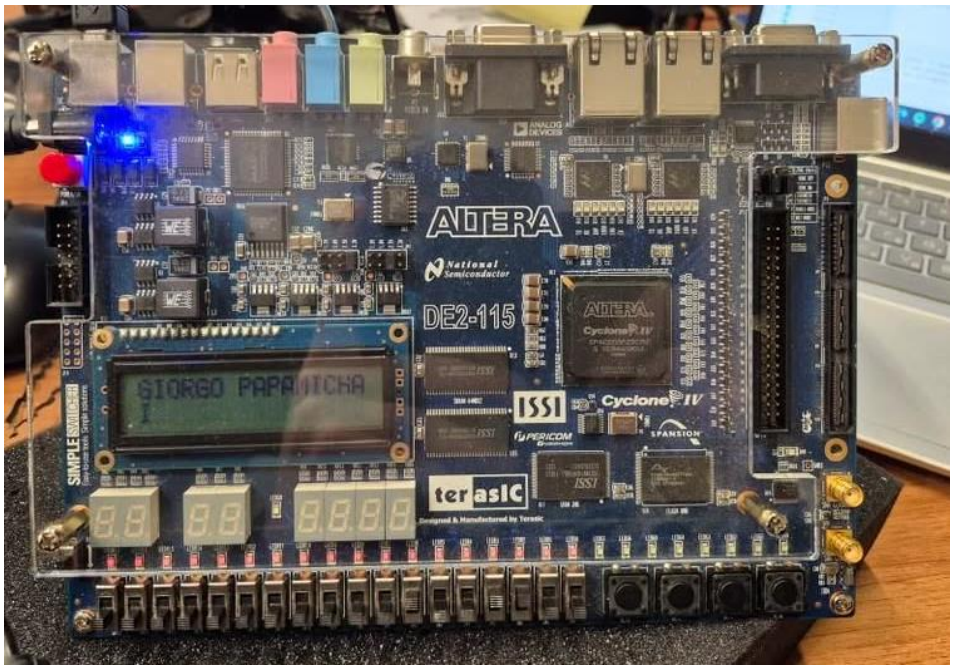
```
; ************************************************************
; Project: Name Display & Logic Operations
; Author:  GIORGO PAPAMICHAIL
; ************************************************************

*d1 : b00001111
*d2 : xAf

add A

; G  I  O  R  G  O      P  A  P  A  M  I  C  H  A  I  L
; 47 49 4F 52 47 4F 20 50 41 50 41 4D 49 43 48 41 49 4C

; Store 'G' to address d1
MOV A, x47
mov *d1, A

; Store 'I' minus 5 to address d2
mov ACC1, b01001001
mov acc2, 5

sub A

mov *d2, A


; Print character G
mov C, *d1
PRT

; If jump fails, we print dump
mov C, x47
jmp save_the_print
PRT

; Take d2 and check if it's lower than 'I'
; If it is, add 5 and print it, else terminate
- save_the_print
;cmp *d2, b01001001
;jb print_I

;PRT

;jmp terminate
- print_I
mov ACC1, *d2
mov ACC2, 5
```

```
add B

mov c, b

prt

; Shift 'O' Left and Right then print
mov ACC1, x4f

shl ACC1, 1 ; Result: 10011110
shr acc1, 1 ; Result: 01001111

mov acc2, 0

add C

prt

; Now we have "GIO"


; If we have an overflow, print 'R'
mov A, 255
mov B, 255

add A, B; Now A is 11111110 (after overflow)

jc overflowmet

mov c, x49
prt

mov c, x30
prt


- overflowmet
; A = 11111110, 'R' = 01010010

;            11111110
mov acc1, xAC


xor A, acc1


mov C, A
prt

; Print 'G' that was stored at d1
```

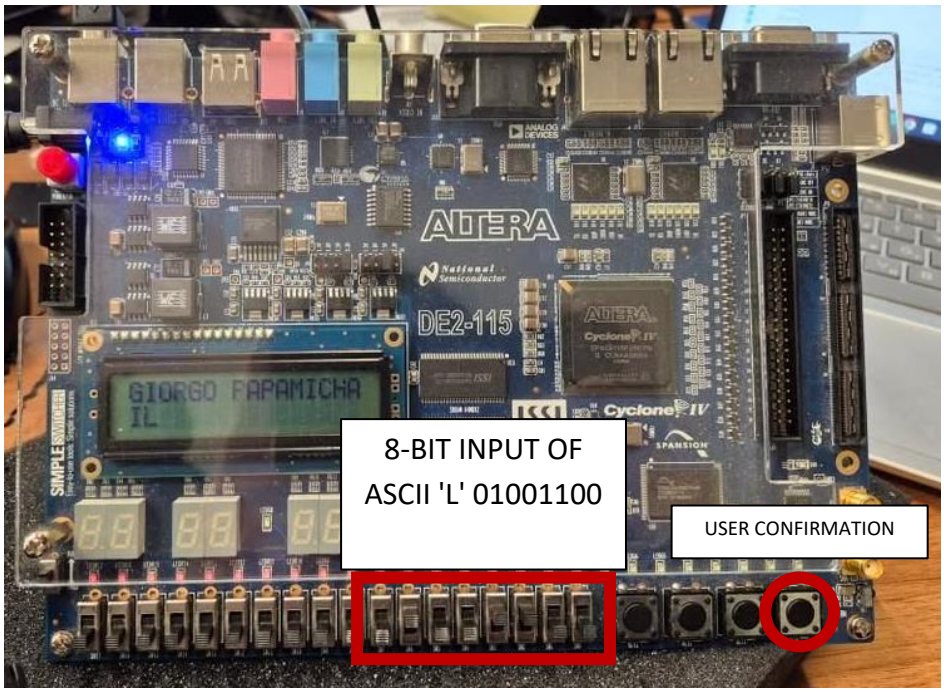
```
mov C, *d1
mov C, *d1
prt

; Print 'O' by performing AND with FF and 'O'
mov acc1, b11111111
mov acc2, b01001111

and c

prt

; Printing space character (x20)
mov A, x02
shl A, 4

mov C, a
mov C, a

prt


; Loop attempt to print "PAPA"

mov A, 0

jmp printP


mov C, x30
prt
mov C, x30
prt
mov C, x30
prt
mov C, x30
prt
mov C, x30
prt


- printP
mov C, x50
prt

jmp printA


- printA
mov C, x41
```

```
prt

inc a, a

cmp A, 1
jbe printP

; Print M
mov C, x4d
prt

; Print I
mov C, x49
prt

; Print C
mov C, x43
prt

; Print H
mov C, x48
prt

; Print A
mov C, x41
prt

; Print I
mov C, x49
prt


; User must input/confirm their name
in c
mov c,c
prt

-terminate
HLT
```

Αποτέλεσμα Πριν Το USER INPUT



Αποτέλεσμα Μετά Το USER INPUT



10. Περιορισμοί

Η τρέχουσα αρχιτεκτονική του 8-bit επεξεργαστή, αν και λειτουργική για εκπαιδευτικούς σκοπούς και βασικές διεργασίες, παρουσιάζει τους εξής περιορισμούς:

- **Περιορισμένη Μνήμη RAM:** Ο επεξεργαστής διαθέτει 256 διευθύνσεις RAM, καθεμία από τις οποίες αποθηκεύει ένα byte (0x00 έως 0xFF). Αυτό περιορίζει το μέγεθος των δεδομένων που μπορούν να υποβληθούν σε επεξεργασία ταυτόχρονα.
- **Πρόσβαση σε Καταχωρητές:** Παρόλο που υπάρχουν 8 καταχωρητές συνολικά, ο προγραμματιστής έχει άμεση πρόσβαση μόνο στους 5 (A, B, ACC1, ACC2, C). Οι υπόλοιποι (IR, PC, SR) προορίζονται αποκλειστικά για εσωτερική χρήση από την Control Unit.
- **Σύστημα Flags:** Χρησιμοποιούνται μόνο 2 bits για Carry και Zero flag. Η απουσία Negative ή Overflow flags καθιστά τη διαχείριση προσήμων και πολύπλοκων αριθμητικών πράξεων πιο δύσκολη.
- **Συγχρονισμός I/O:** Η είσοδος δεδομένων απαιτεί την επιβεβαίωση (confirm) από τον χρήστη, γεγονός που αναγκάζει την Control Unit να παραμένει σε κατάσταση αναμονής μέχρι να λάβει το σήμα, σταματώντας την εκτέλεση του υπόλοιπου προγράμματος.
- **Απουσία Stack Pointer (SP):** Ο επεξεργαστής δεν διαθέτει καταχωρητή Stack Pointer ή μηχανισμό στοίβας (Stack). Αυτό σημαίνει ότι δεν υποστηρίζονται εντολές PUSH/POP και η κλήση υπορουτινών είναι αδύνατη, καθώς δεν υπάρχει αυτόματος τρόπος αποθήκευσης της διεύθυνσης επιστροφής.

Μελλοντικές Επεκτάσεις (Future Expansions)

Για την εξέλιξη του επεξεργαστή σε ένα πιο ισχυρό υπολογιστικό σύστημα, προτείνονται οι παρακάτω βελτιώσεις:

- **Προσθήκη Stack Pointer (SP):** Η υλοποίηση ενός καταχωρητή 8-bit για SP θα επέτρεπε τη χρήση της RAM ως στοίβας. Αυτό θα επέτρεπε την εισαγωγή εντολών CALL και RET, καθιστώντας δυνατή τη χρήση συναρτήσεων και αναδρομικών αλγορίθμων.
- **Διεύρυνση Address Bus:** Η αύξηση των bits διευθυνσιοδότησης (π.χ. από 8-bit σε 16-bit) θα επέτρεπε την πρόσβαση σε πολύ μεγαλύτερο εύρος μνήμης RAM, πέρα από τις τρέχουσες 256 θέσεις.
- **Εμπλουτισμός Instruction Set (ISA):** Προσθήκη εντολών για απευθείας πράξεις μεταξύ καταχωρητών και μνήμης (Memory-to-Register) χωρίς την ενδιάμεση χρήση του ACC1.

- **Αναβάθμιση Status Register:** Προσθήκη επιπλέον flags, όπως το Negative flag (για πράξεις με πρόσημο) και το Overflow flag (για τον εντοπισμό λαθών σε πρόσθεση/αφαίρεση συμπληρώματος ως προς 2).
- **Interrupt Controller:** Η προσθήκη ενός συστήματος διακοπών (interrupts) θα επέτρεπε στον επεξεργαστή να ανταποκρίνεται άμεσα σε εξωτερικά σήματα (όπως το user input) χωρίς να χρειάζεται να "παγώνει" η εκτέλεση σε κατάσταση αναμονής.